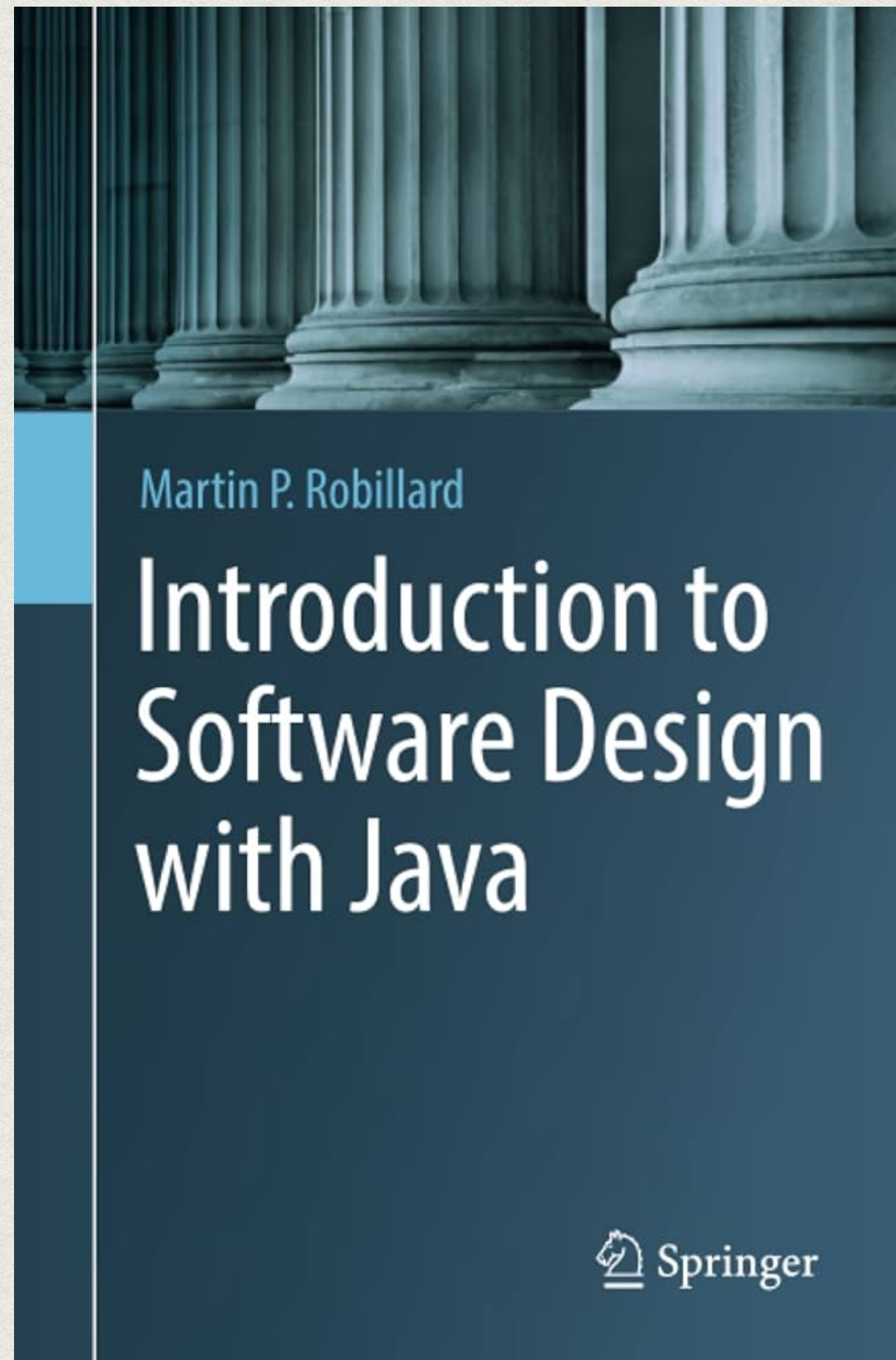




M1 MIAGE / MAPC TYPES ET INTERFACES

Pascal POIZAT

RÉFÉRENCES



Conception Logicielle
Site web du cours INF5153 de l'UQAM

- [Retourner à la page d'accueil](#)

Introduction

Notes de cours

- [Introduction au cours](#)
- [Écosystème de développement](#)
- [Schotten Totten](#)

Capsules Vidéos

Introduction au cours

Introduction du cours de Conception Logicielle [IN...]

Partager

Génie logiciel : Conception

Regarder sur YouTube

Sébastien Mosser - INF5153
Chapitre 0 - Capsule 1
Automne 2020

ace

crédits photos: Pixabay

INTERFACE

- on a vu précédemment comment bien définir des classes
bonne **encapsulation**
- mais comment **inter-agissent** ces classes (les objets de ces classes) ?
→ notion d'**interface**
attention : le terme est surchargé en informatique

EXAMPLE

```
public class Deck {  
    private List<Card> aCards;  
    public Deck() {...}  
    public void shuffle() {...}  
    public Card draw() {...}  
    public boolean isEmpty() {...}  
}
```

```
public class Client { private Deck aDeck = new Deck(); ... }
```

```
public static List<Card> drawCards(Deck pDeck, int pNumber) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < pNumber && !pDeck.isEmpty(); i++) {  
        result.add(pDeck.draw());  
    }  
    return result;  
}
```


EXEMPLE

- **couplage fort**

Client → Deck

- dépendance

à une implémentation (Deck)
plutôt qu'à une abstraction

- il peut y avoir
plusieurs façons
de gérer et tirer des cartes

```
public class Deck {  
    private List<Card> aCards;  
    public Deck() {...}  
    public void shuffle() {...}  
    public Card draw() {...}  
    public boolean isEmpty() {...}  
}
```

```
public class Client { private Deck aDeck = new Deck(); ... }
```

```
public static List<Card> drawCards(Deck pDeck, int pNumber) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < pNumber && !pDeck.isEmpty(); i++) {  
        result.add(pDeck.draw());  
    }  
    return result;  
}
```


EXEMPLE CORRIGÉ

```
public interface CardSource {
    Card draw(); // pre: !isEmpty(), return: next card
    boolean isEmpty(); // return: true if no card in the source
}

public class Deck implements CardSource {
    private List<Card> aCards;
    public Deck() {...}
    public void shuffle() {...}
    @Override public Card draw() {...}
    @Override public boolean isEmpty() {...}
}

public class Client { private CardSource aSource = new Deck(); ...}

public static List<Card> drawCards(CardSource pSource, int pNumber) {
    List<Card> result = new ArrayList<>();
    for (int i = 0; i < pNumber && !pSource.isEmpty(); i++) {
        result.add(pSource.draw());
    }
    return result;
}
```


EXEMPLE CORRIGÉ

- définition d'une **interface**
couplage faible

Client → CardSource

- permet :
vérification de typage
(classe ↔ interface ↔ client)

polymorphisme

et extensibilité

(CardSource prend ≠ formes)

- penser à :
commentaires et contrats

```
public interface CardSource {  
    Card draw(); // pre: !isEmpty(), return: next card  
    boolean isEmpty(); // return: true if no card in the source  
}
```

contrats

```
public class Deck implements CardSource {  
    private List<Card> aCards;  
    public Deck() {...}  
    public void shuffle() {...}  
    @Override public Card draw() {...}  
    @Override public boolean isEmpty() {...}  
}
```

```
public class Client { private CardSource aSource = new Deck(); ...}
```

```
public static List<Card> drawCards(CardSource pSource, int pNumber) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < pNumber && !pSource.isEmpty(); i++) {  
        result.add(pSource.draw());  
    }  
    return result;  
}
```


ITERATOR

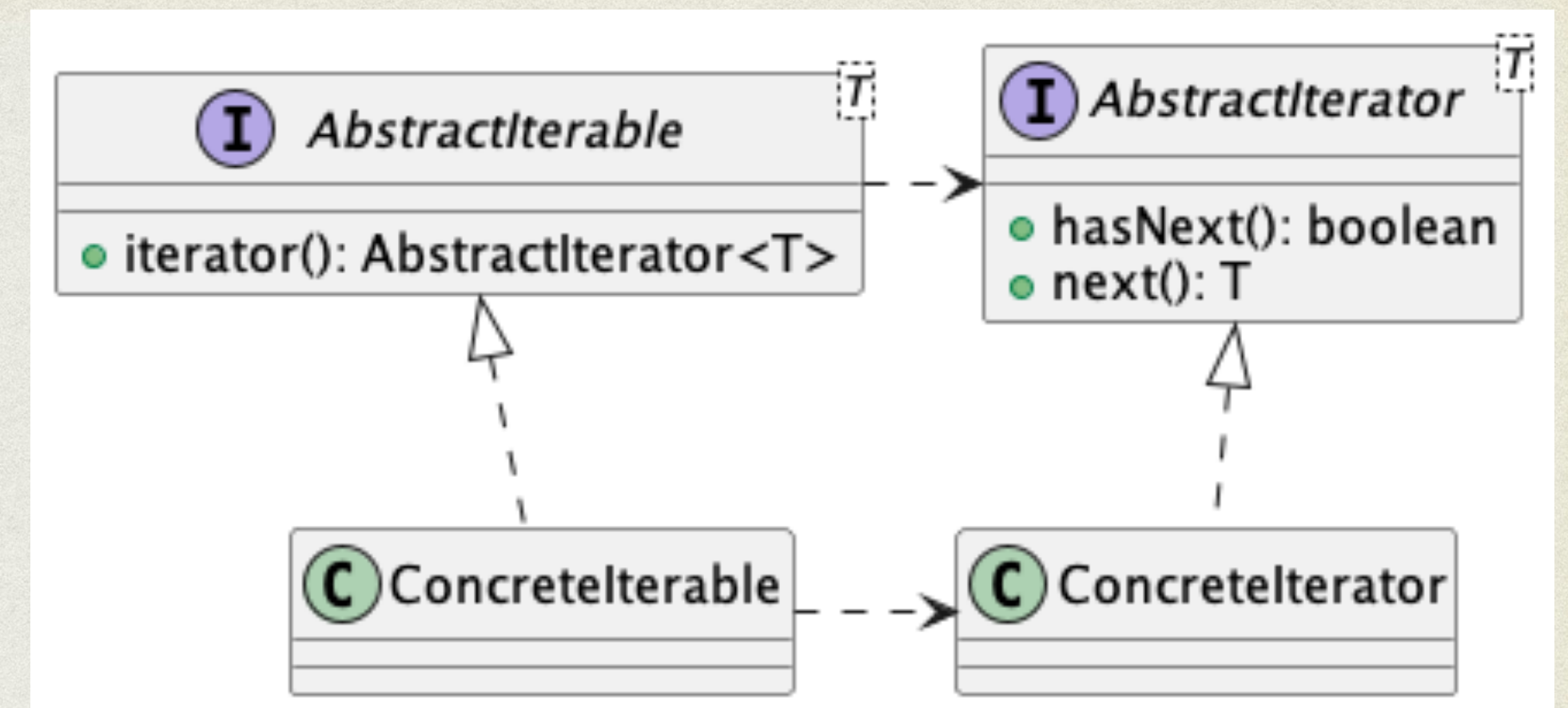


ITERATOR

- comment donner **accès** à une collection en **respectant** encapsulation et masquage d'information ?
- on a vu des solutions (cf cours encapsulation)
- autre solution, se demander quel est le **besoin** itérer sur les éléments ? patron **Iterator**

ITERATOR

- comment donner **accès** à une collection en **respectant** encapsulation et masquage d'information ?
- on a vu des solutions (cf cours encapsulation)
- autre solution, se demander quel est le **besoin** itérer sur les éléments ? patron **Iterator**



ITERATOR

- comment donner **accès** à une collection en **respectant** encapsulation et masquage d'information ?

- on a vu des solutions (cf cours encapsulation)

- autre solution, se demander quel est le **besoin** itérer sur les éléments ? patron **Iterator**

@Override

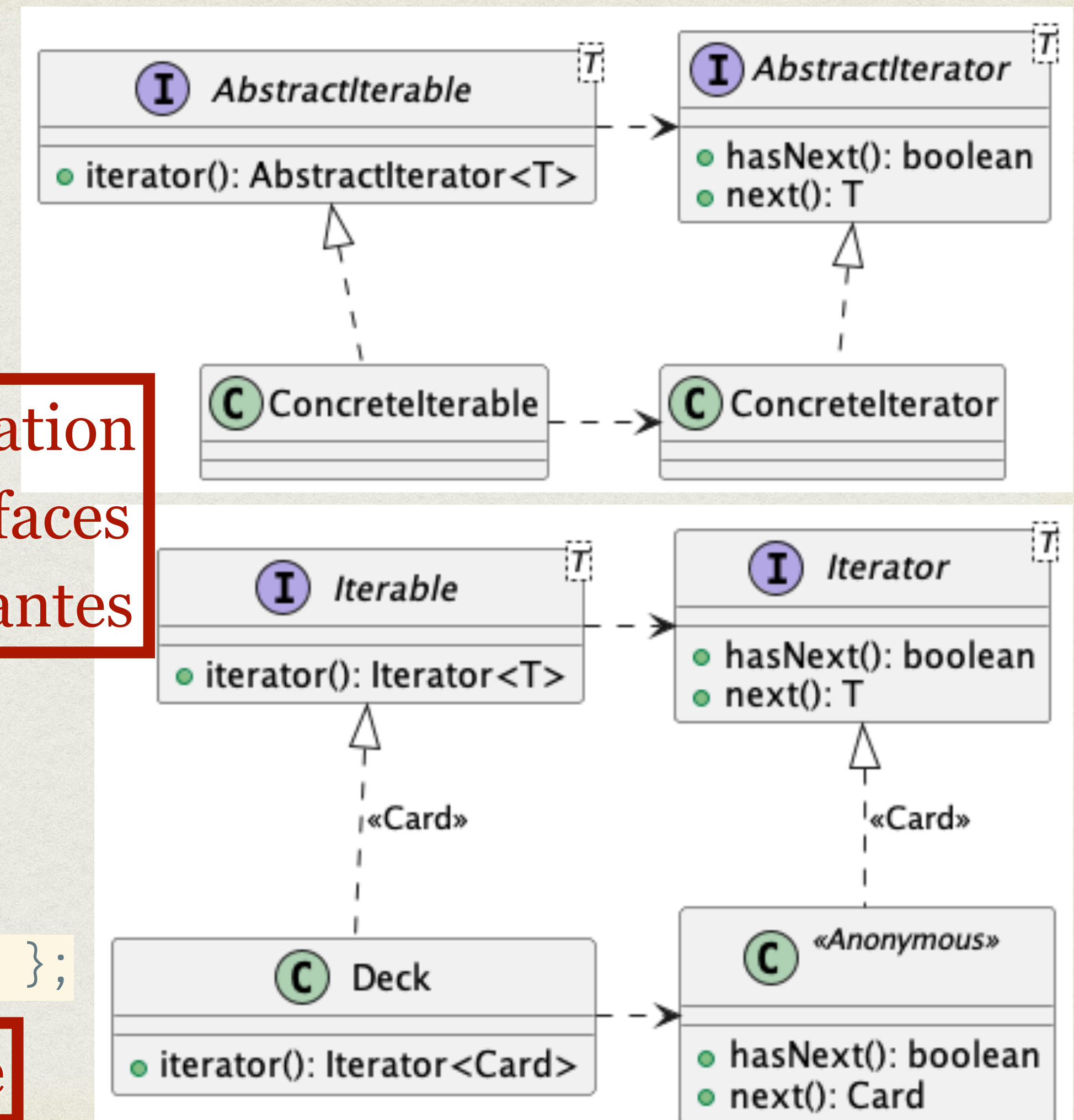
```
public Iterator<Card> iterator() { return aCards.iterator(); };
```

```
Iterable<Card> d = new Deck();
```

```
for(Card c: d) { System.out.println(c); }
```

utilisation
interfaces
existantes

bénéfice syntaxe dédiée



PÉRIMÈTRE D'INTERFACE

```
public interface CardSource {  
    Card draw();           // pre: !isEmpty(), return: next card  
    boolean isEmpty();     // return: true if no card in the source  
}
```

```
public class Deck implements CardSource {  
    private List<Card> aCards;  
    public Deck() {...}  
    public void shuffle() {...}  
    public Iterator<Card> iterator() {...}  
    @Override public Card draw() {...}  
    @Override public boolean isEmpty() {...}  
}
```

```
public class Client { private CardSource aSource = new Deck(); ...}
```

```
public static List<Card> drawCards(CardSource pSource, int pNumber) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < pNumber && !pSource.isEmpty(); i++) {  
        result.add(pSource.draw());  
    }  
    return result;  
}
```


PÉRIMÈTRE D'INTERFACE

- attention au **périmètre** lors de la définition des interfaces
- **cohérence** :
shuffle et iterator
vont-t-ils « **avec** »
draw et isEmpty ?

```
public interface CardSource {  
    Card draw();          // pre: !isEmpty(), return: next card  
    boolean isEmpty();    // return: true if no card in the source  
}
```

```
public class Deck implements CardSource {  
    private List<Card> aCards;  
    public Deck() {...}  
    public void shuffle() {...}  
    public Iterator<Card> iterator() {...}  
    @Override public Card draw() {...}  
    @Override public boolean isEmpty() {...}  
}
```

```
public class Client { private CardSource aSource = new Deck(); ...}
```

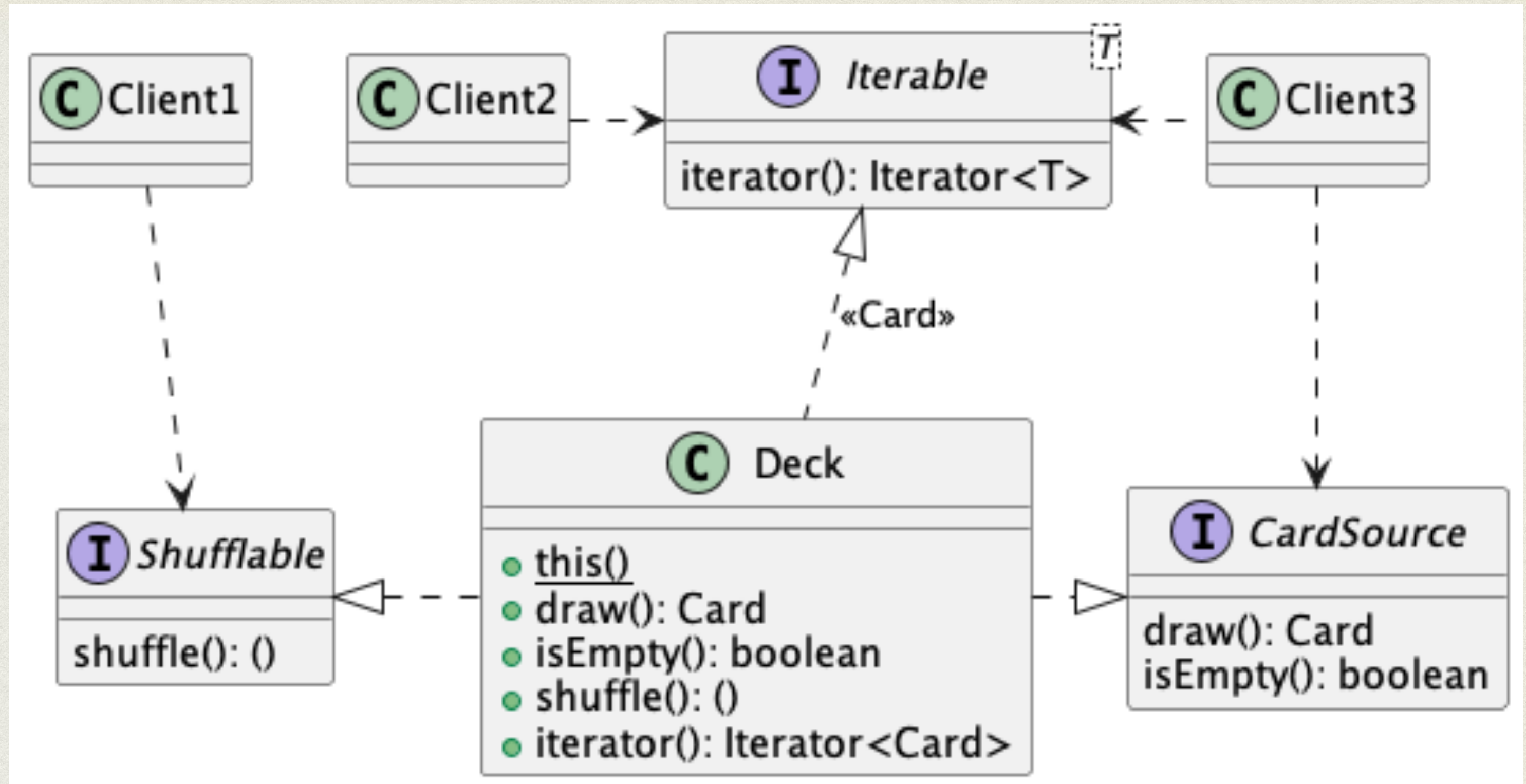
```
public static List<Card> drawCards(CardSource pSource, int pNumber) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < pNumber && !pSource.isEmpty(); i++) {  
        result.add(pSource.draw());  
    }  
    return result;  
}
```


PÉRIMÈTRE D'INTERFACE

- principe **ISP** de SOLID
- dépendre **uniquement**
des services
dont on a **besoin**
- une implémentation
peut réaliser
plusieurs contrats
- promouvoir la **séparation des préoccupations**

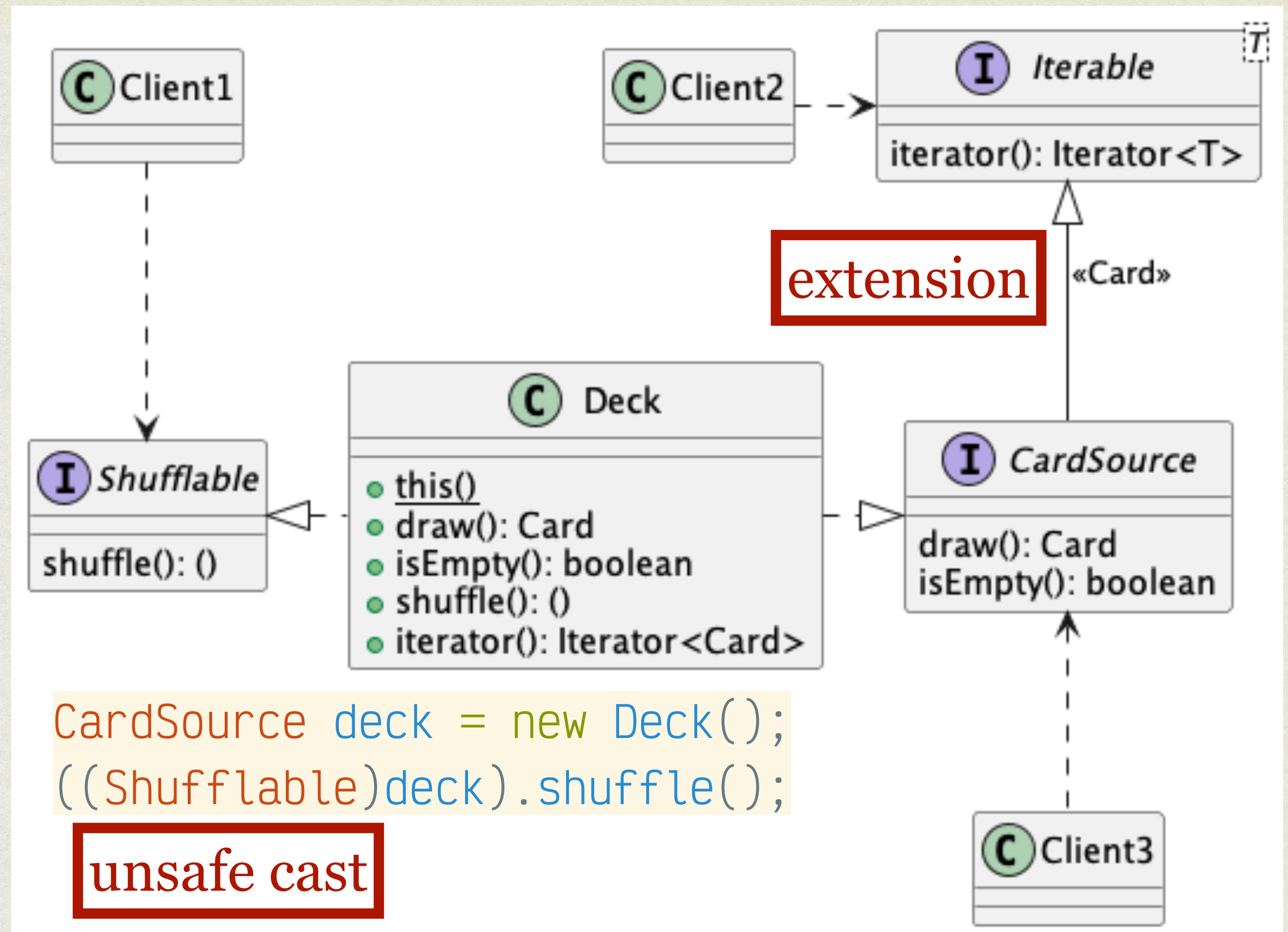
PÉRIMÈTRE D'INTERFACE

- principe **ISP** de SOLID
- dépendre **uniquement** des services dont on a **besoin**
- une implémentation peut réaliser **plusieurs** contrats
- promouvoir la **séparation des préoccupations**



EXTENSION D'INTERFACE

- on peut **étendre** les interfaces
- dépend de l'usage par les clients
utilisation B $\Rightarrow$ utilisation A ?
alors **extension B $\triangleright$ A**
- **problème** :
ici si on veut mélanger puis tirer
on risque un **unsafe cast**
 $\rightarrow$ parfois revenir sur choix ISP



COMPARAISON ET TRI (V_1)

- supposons que l'on veuille **trier** les cartes d'un deck

COMPARAISON ET TRI (V1)

- supposons que l'on veuille **trier** les cartes d'un deck

```
public void sort() {  
    Collections.sort(aCards);  
}
```


COMPARAISON ET TRI (V1)

- supposons que l'on veuille **trier** les cartes d'un deck

```
public void sort() {  
    Collections.sort(aCards);  
}
```

The method sort(List<T>) in the type Collections
is not applicable for the arguments
(List<Card>) Java(67108979)

```
<T> void java.util.Collections.sort(List<T> list)
```


COMPARAISON ET TRI (V1)

- supposons que l'on veuille **trier** les cartes d'un deck

```
public void sort() {  
    Collections.sort(aCards);  
}
```

The method sort(List<T>) in the type Collections
is not applicable for the arguments
(List<Card>) Java(67108979)

```
<T> void java.util.Collections.sort(List<T> list)
```

- **contrat** : les cartes doivent être **Comparable**

COMPARAISON ET TRI (V1)

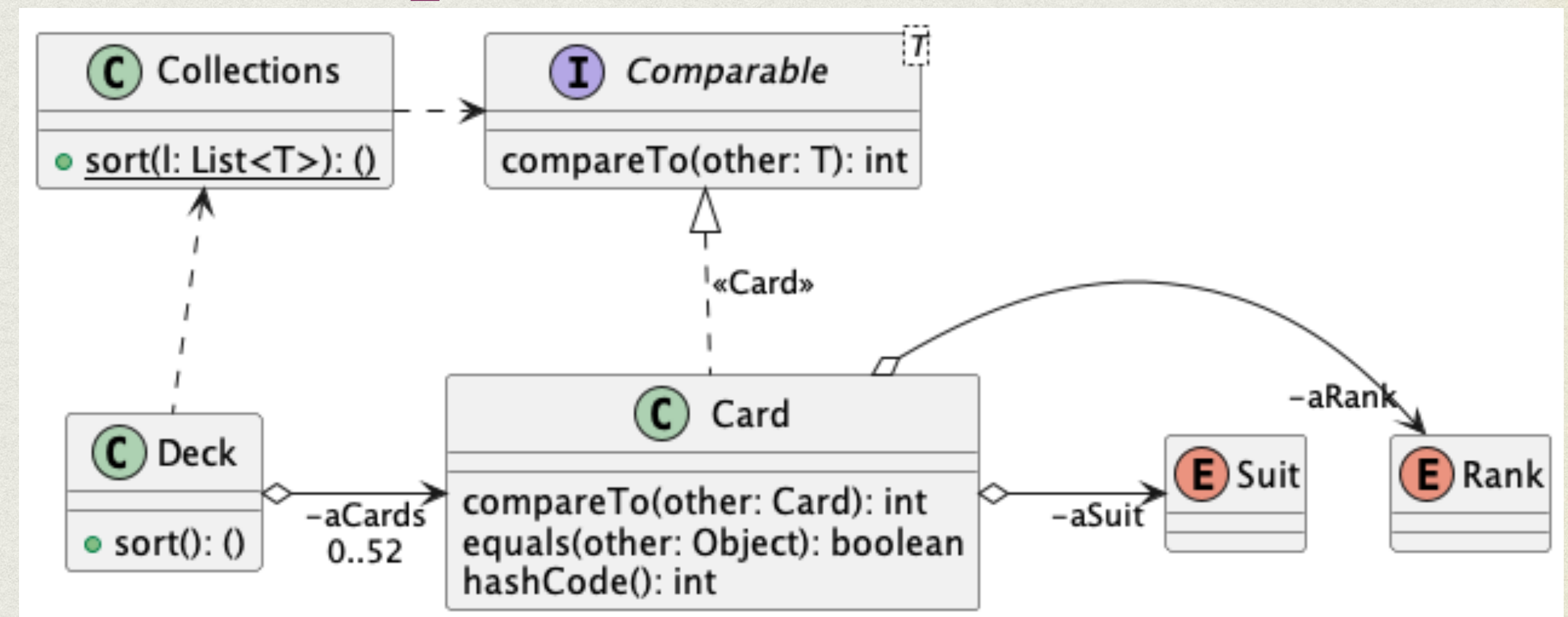
- supposons que l'on veuille **trier** les cartes d'un deck

```
public void sort() {  
    Collections.sort(aCards);  
}
```

The method sort(List<T>) in the type Collections is not applicable for the arguments (List<Card>) Java(67108979)

```
<T> void java.util.Collections.sort(List<T> list)
```

- contrat** : les cartes doivent être **Comparable**



```
@Override
```

```
public int compareTo(Card o) {  
    return this.aRank.ordinal() - o.aRank.ordinal();  
}
```


COMPARAISON ET TRI (V1)

- supposons que l'on veuille **trier** les cartes d'un deck

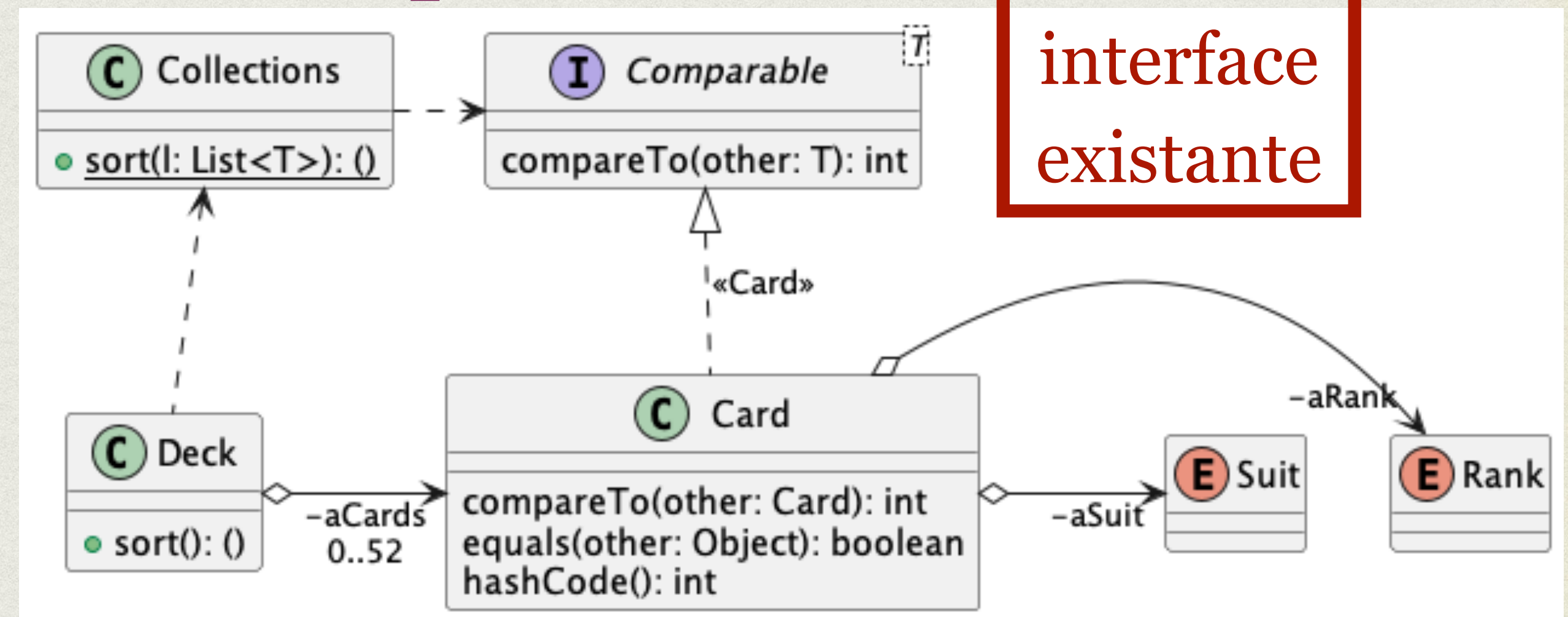
```
public void sort() {  
    Collections.sort(aCards);  
}
```

The method sort(List<T>) in the type Collections is not applicable for the arguments (List<Card>) Java(67108979)

```
<T> void java.util.Collections.sort(List<T> list)
```

- contrat** : les cartes doivent être **Comparables**

utilisation
interface
existante



attention : lien equals/hashCode/compareTo

@Override

```
public int compareTo(Card o) {  
    return this.aRank.ordinal() - o.aRank.ordinal();  
}
```


INTERLUDE : RÈGLES

- bien **lire la documentation des API** que l'on utilise
notons

$o1 \equiv o2 \iff o1.equals(o2)$, $\#o \iff o.hashCode()$,

$o1 < o2 \iff o1.compareTo(o2) < 0$, $o1 > o2 \iff o1.compareTo(o2) > 0$, $o1 \sim o2 \iff o1.compareTo(o2) = 0$

- **requis** (entre autres) :

(=1) $o \equiv o$

(=2) $o1 \equiv o2 \Rightarrow o2 \equiv o1$

(=3) $o1 \equiv o2 \wedge o2 \equiv o3 \Rightarrow o1 \equiv o3$

(#1) $o1 \equiv o2 \Rightarrow \#o1 = \#o2$

(<1) $o1 < o2 \iff o2 > o1$

(<2) $o1 < o2 \wedge o2 < o3 \Rightarrow o1 < o3$

- **non requis** :

$\text{not}(o1 \equiv o2) \Rightarrow \text{not}(\#o1 = \#o2)$

recommandé mais non requis (documenter) :

$o1 \sim o2 \iff o1 \equiv o2$

COMPARAISON ET TRI (V2)

- et si on a plusieurs façons de trier les cartes ?
 - par rang (A 2 3 ... 10 V D R)
 - par rang (2 3 ... 10 V D R A)
 - par couleur
 - belote atout (7 8 D R 10 A 9 V)
 - belote hors atout (7 8 9 V D R 10 A)
- utiliser un **flag** dans Card ?
(paramètre le choix de tri)

COMPARAISON ET TRI (V2)

- et si on a plusieurs façons de trier les cartes ?
 - par rang (A 2 3 ... 10 V D R)
 - par rang (2 3 ... 10 V D R A)
 - par couleur
 - belote atout (7 8 D R 10 A 9 V)
 - belote hors atout (7 8 9 V D R 10 A)
- utiliser un **flag** dans Card ?
(paramètre le choix de tri)

```
public class Card implements Comparable<Card> {  
    private Suit aSuit;  
    private Rank aRank;  
    private CompareMode compareMode = CompareMode.RANK;  
  
    public enum CompareMode { RANK, SUIT };  
  
    public void setCompareMode(CompareMode compareMode) {  
        this.compareMode = compareMode; }  
    @Override  
    public int compareTo(Card o) {  
        return switch(this.compareMode) {  
            case RANK → this.aRank.ordinal()-o.aRank.ordinal();  
            case SUIT → this.aSuit.ordinal()-o.aSuit.ordinal();  
        };  
    }  
}
```


COMPARAISON ET TRI (V2)

- et si on a plusieurs façons de trier les cartes ?
 - par rang (A 2 3 ... 10 V D R)
 - par rang (2 3 ... 10 V D R A)
 - par couleur
 - belote atout (7 8 D R 10 A 9 V)
 - belote hors atout (7 8 9 V D R 10 A)
- utiliser un **flag** dans Card ?
(paramètre le choix de tri)

```
public class Card implements Comparable<Card> {  
    private Suit aSuit;  
    private Rank aRank;  
    private CompareMode compareMode = CompareMode.RANK;
```

```
public enum CompareMode { RANK, SUIT },
```

perte immutabilité

```
public void setCompareMode(CompareMode compareMode) {  
    this.compareMode = compareMode; }
```

```
@Override
```

```
public int compareTo(Card o) {
```

```
    return switch(this.compareMode) {
```

```
        case RANK → this.aRank.ordinal()-o.aRank.ordinal();
```

```
        case SUIT → this.aSuit.ordinal()-o.aSuit.ordinal();
```

```
    };
```

Switch Statement†

```
}
```

```
}
```


COMPARAISON ET TRI (V2)

- déplacer la **responsabilité** de la comparaison
- un **Comparator** par tri
- différents choix de placement
ici choix dans Deck

```
public class Deck ... {  
    private List<Card> aCards;
```

```
    public Deck() { ... }
```

```
    public static final Comparator<Card> c1 = (c1, c2) → ...;  
    public static final Comparator<Card> c2 = (c1, c2) → ...;
```

```
    public void sort(int choix) {  
        switch (choix) {  
            case 0 → Collections.sort(aCards, c1);  
            case 1 → Collections.sort(aCards, c2);  
            default → Collections.sort(aCards, c2);  
        }  
    }  
}
```

function objects
functional interfaces

Switch Statement†

COMPARAISON ET TRI (V2)

- est-ce le meilleur choix ?
si Card a des attributs privés ?
- création de Comparators
dans Card à l'aide de
factory methods
(= méthodes de fabrication
d'objets)

COMPARAISON ET TRI (V2)

factory methods

- est-ce le meilleur choix ?
si Card a des attributs privés ?
- création de Comparators
dans Card à l'aide de
factory methods
(= méthodes de fabrication
d'objets)

```
public class Card {
```

```
...
```

```
public static Comparator<Card> rankComparator()  
{ return (card1, card2) → ...; }
```

```
public static Comparator<Card> suitComparator()  
{ return (card1, card2) → ...; }
```

```
}
```

```
public class Deck ... {
```

```
...
```

```
public void sort(int choix) {
```

```
    switch (choix) {
```

```
        case 0 → Collections.sort(aCards,  
                                   Card.rankComparator());
```

```
        case 1 → Collections.sort(aCards,  
                                   Card.suitComparator());
```

```
        default → Collections.sort(aCards, ...);
```

```
    }}
```

```
}
```


COMPARAISON ET TRI (V2)

factory methods

- est-ce le meilleur choix ?
si Card a des attributs privés ?

```
public class Card {  
    ...  
    public static Comparator<Card> rankComparator()  
        { return (card1, card2) → ...; }  
    public static Comparator<Card> suitComparator()  
        { return (card1, card2) → ...; }  
}
```

- création de Comparators
dans Card à l'aide de
factory methods
(= méthodes de fabrication
d'objets)

Switch Statement†

```
public class Deck ... {  
    ...  
    public void sort(int choix) {  
        switch (choix) {  
            case 0 → Collections.sort(aCards,  
                                     Card.rankComparator());  
            case 1 → Collections.sort(aCards,  
                                     Card.suitComparator());  
            default → Collections.sort(aCards, ...);  
        }  
    }  
}
```


TEMPLATE METHOD

- Comment éviter le Switch Statement ?

Switch Statement†

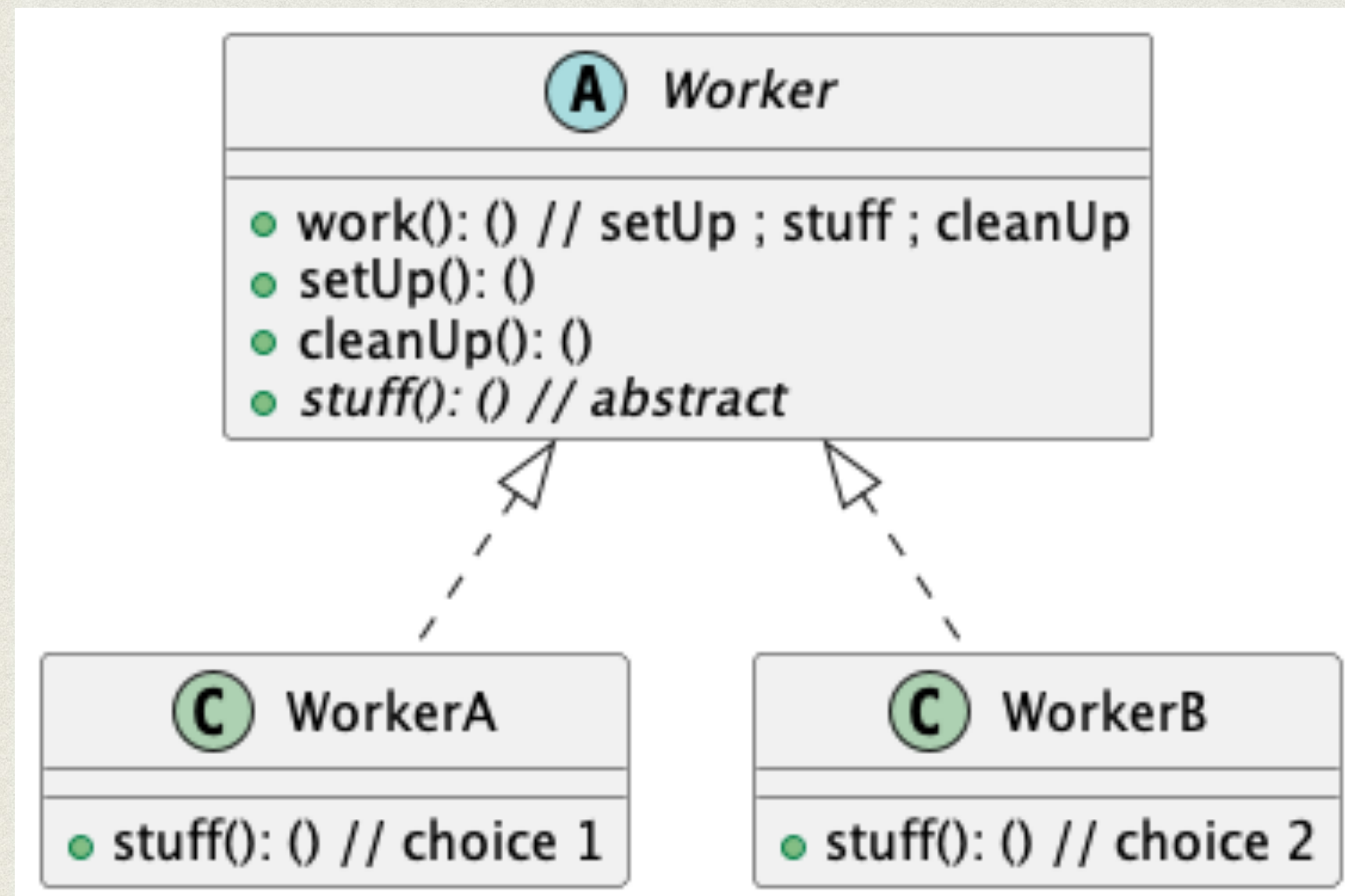
```
public class Worker {  
    public void work(int choice) {  
        setUp();  
        switch (choice) {  
            case 0: choice1(); break;  
            case 1: choice2(); break;  
            default: defaultChoice();  
        }  
        cleanUp();  
    }  
    public void setUp() {}  
    public void cleanUp() {}  
    public void defaultChoice() {}  
    public void choice1() {}  
    public void choice2() {}  
}
```

```
Worker work1 = new Worker();  
work1.work(0);
```

Magic Number†

TEMPLATE METHOD

- Comment éviter le Switch Statement ?



```
Worker work1 = new WorkerA();
work1.work();
```

Switch Statement†

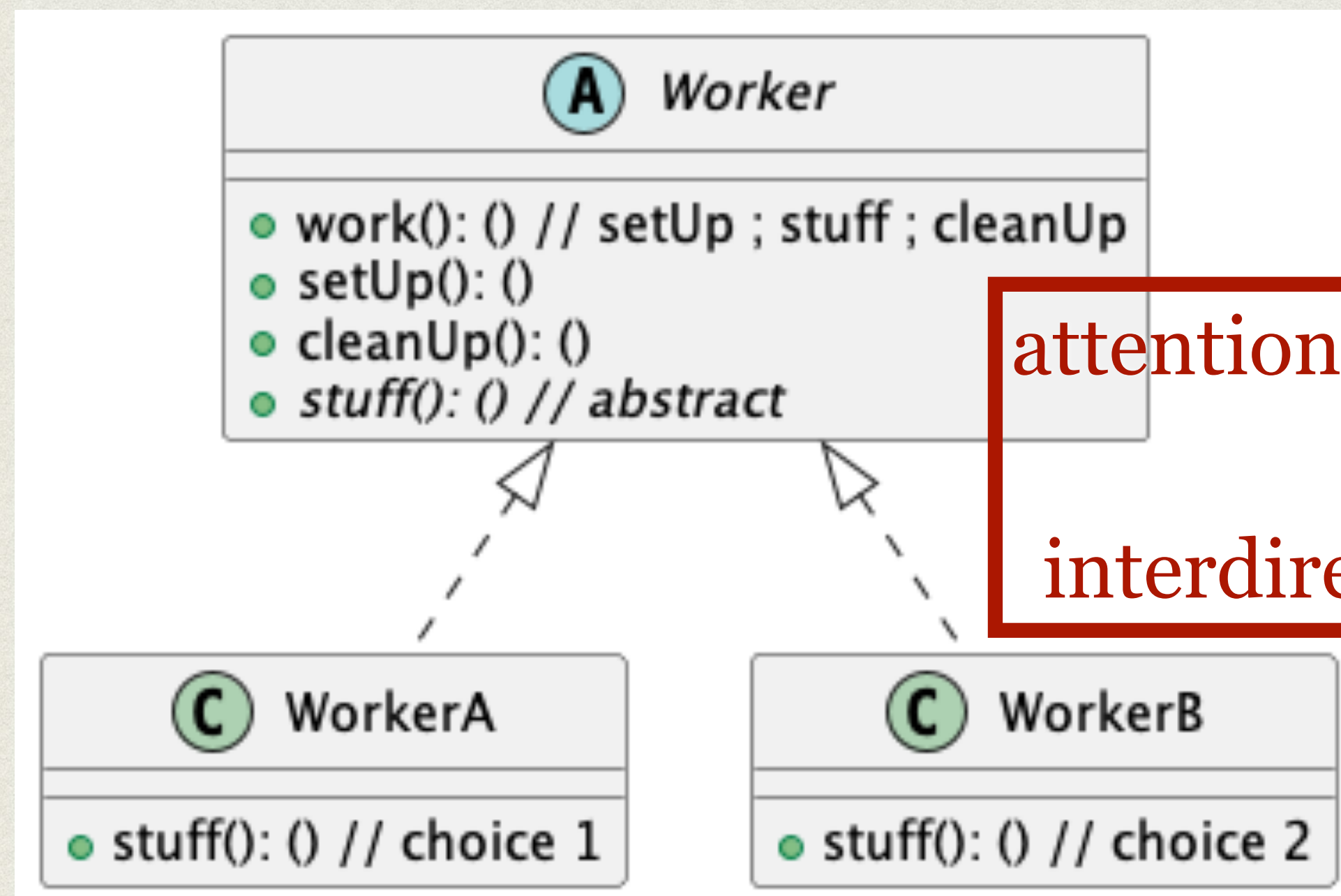
```
public class Worker {
    public void work(int choice) {
        setUp();
        switch (choice) {
            case 0: choice1(); break;
            case 1: choice2(); break;
            default: defaultChoice();
        }
        cleanUp();
    }
    public void setUp() {}
    public void cleanUp() {}
    public void defaultChoice() {}
    public void choice1() {}
    public void choice2() {}
}
```

```
Worker work1 = new Worker();
work1.work(0);
```

Magic Number†

TEMPLATE METHOD

- Comment éviter le Switch Statement ?



attention à documenter
ou
interdire le masquage

```
Worker work1 = new WorkerA();
work1.work();
```

Switch Statement†

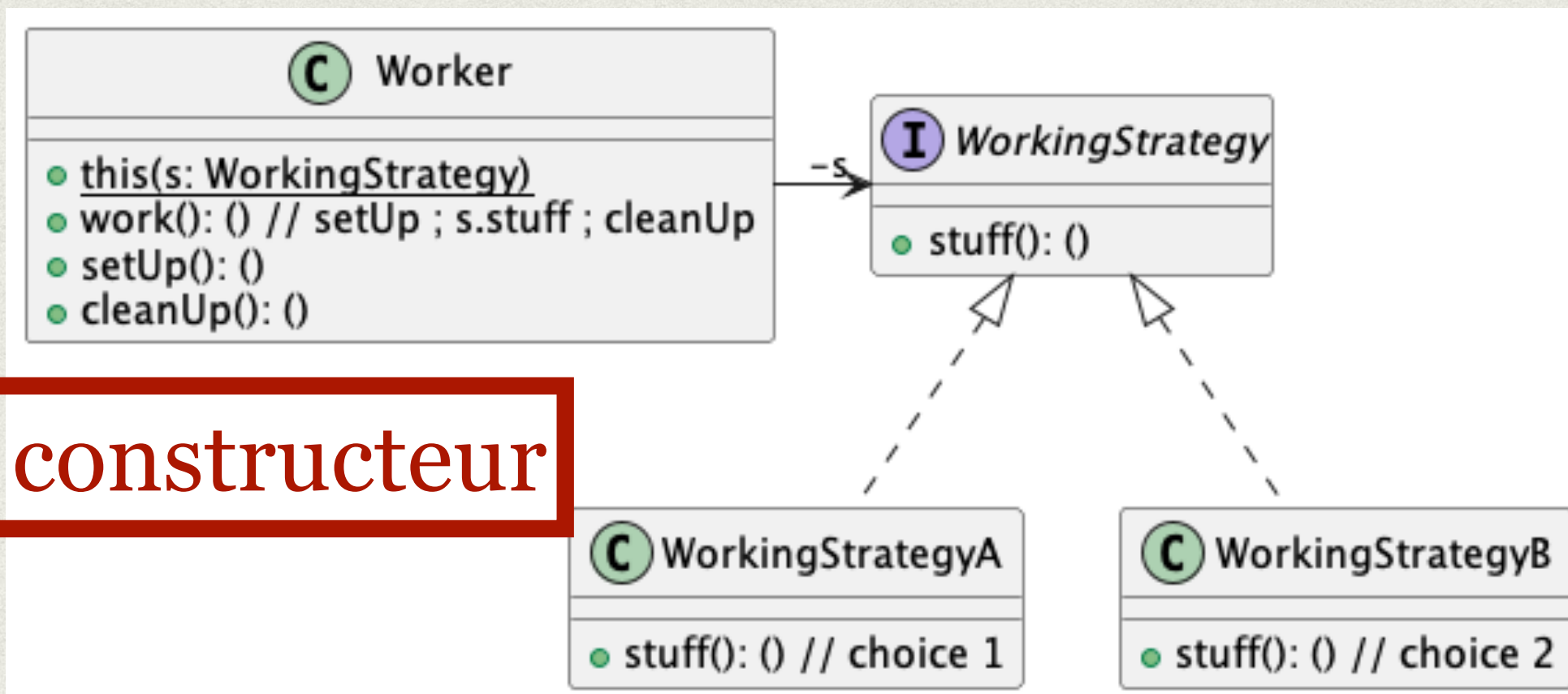
```
public class Worker {
    public void work(int choice) {
        setUp();
        switch (choice) {
            case 0: choice1(); break;
            case 1: choice2(); break;
            default: defaultChoice();
        }
        cleanUp();
    }
    public void setUp() {}
    public void cleanUp() {}
    public void defaultChoice() {}
    public void choice1() {}
    public void choice2() {}
}
```

```
Worker work1 = new Worker();
work1.work(0);
```

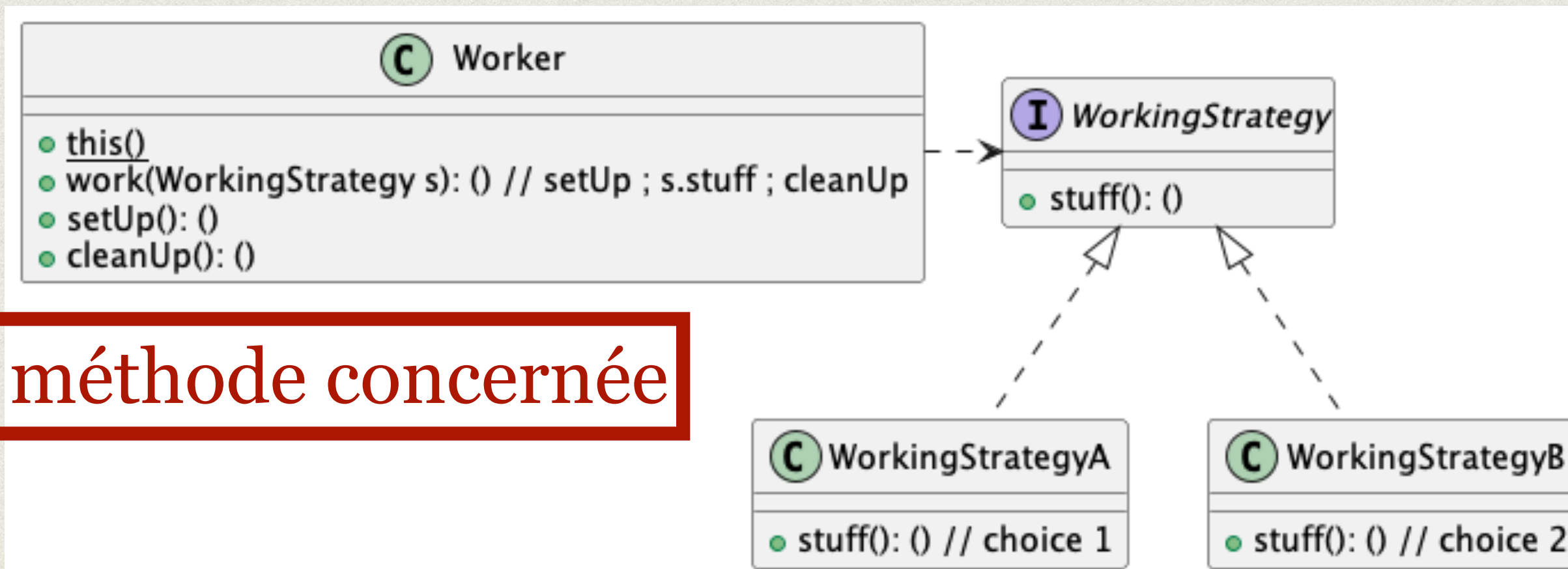
Magic Number†

STRATEGY

- Comment éviter le Switch Statement ?



stratégie et constructeur



stratégie et méthode concernée

Switch Statement†

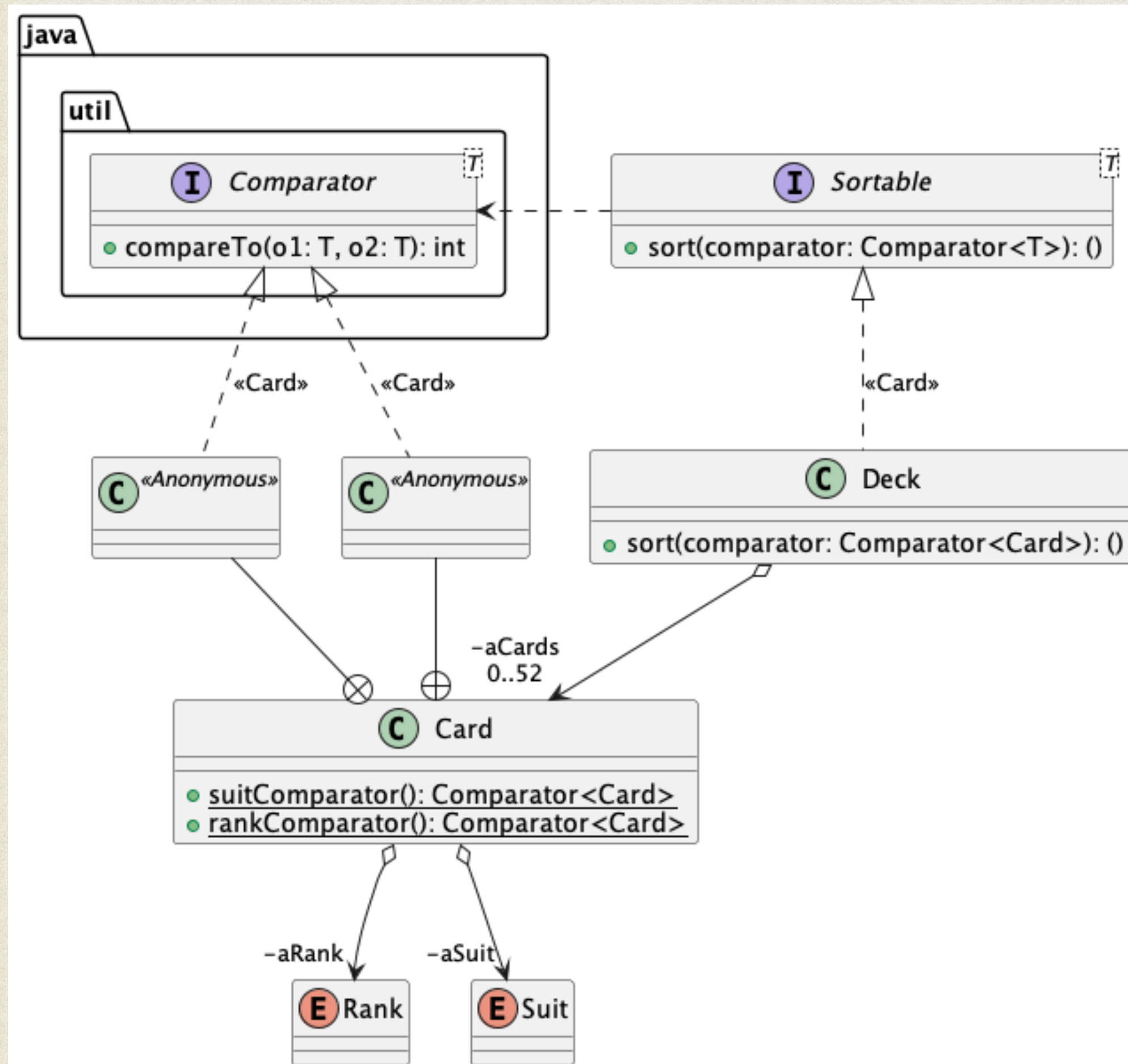
```
public class Worker {
    public void work(int choice) {
        setUp();
        switch (choice) {
            case 0: choice1(); break;
            case 1: choice2(); break;
            default: defaultChoice();
        }
        cleanUp();
    }

    public void setUp() {}
    public void cleanUp() {}
    public void defaultChoice() {}
    public void choice1() {}
    public void choice2() {}
}
```

```
Worker work1 = new Worker();
work1.work(0);
```

Magic Number†

STRATEGY (APPLICATION)



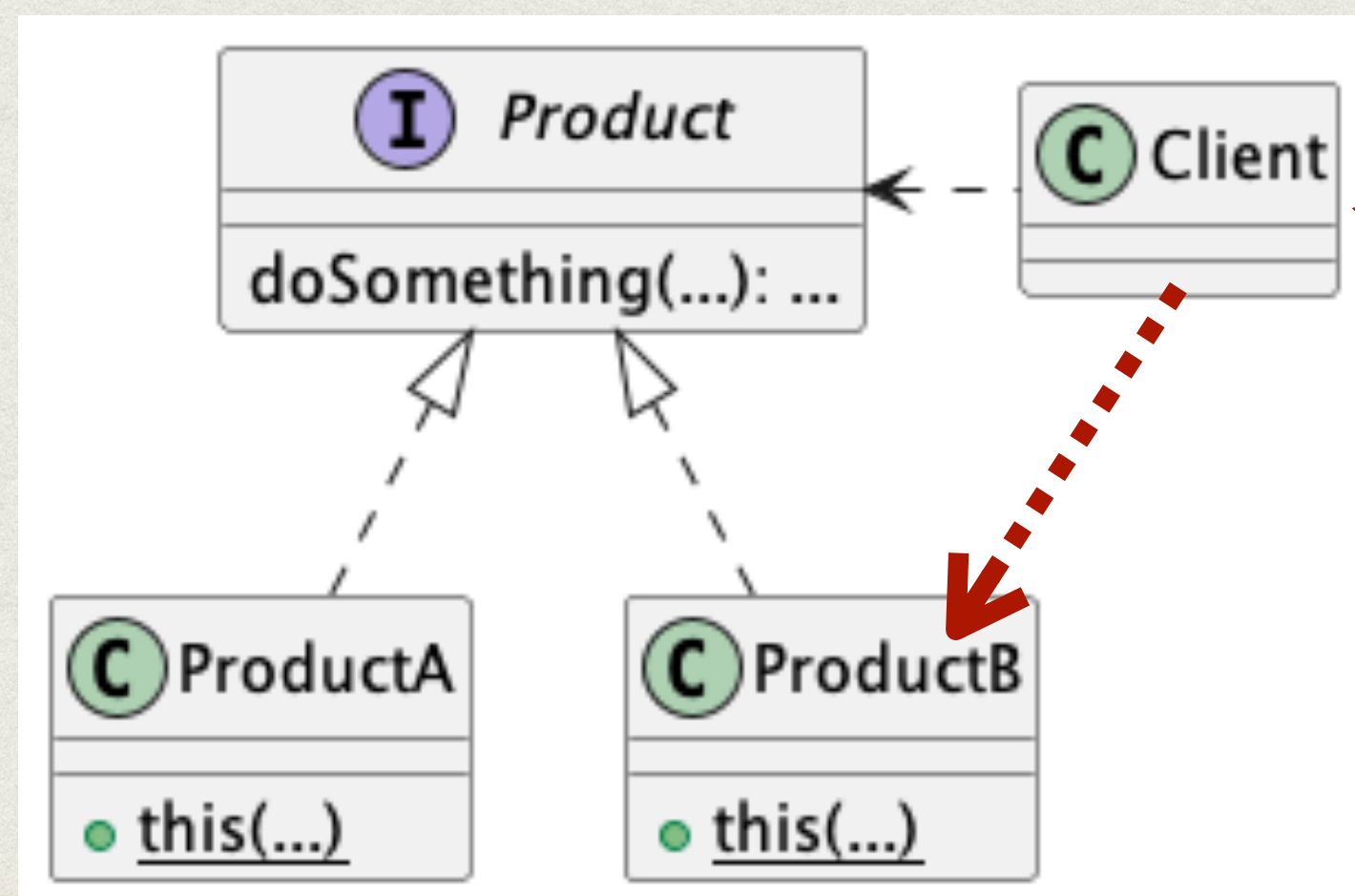
```
public interface Sortable<T> {  
    void sort(Comparator<T> comparator);  
}
```

```
public class Deck implements Sortable<Card>, ... {  
    private List<Card> aCards;  
    public Deck() { ... }  
    ...  
    @Override public void sort(Comparator<Card> comparator) {  
        Collections.sort(aCards, comparator);  
    }  
}
```

```
Sortable<Card> d = new Deck();  
d.sort(Card.rankComparator());
```


CRÉATION D'OBJETS

- le client peut dépendre d'interfaces mais aussi (en même temps) d'**implémentations**
- comment améliorer ce **couplage** ?



constructeur à accès public

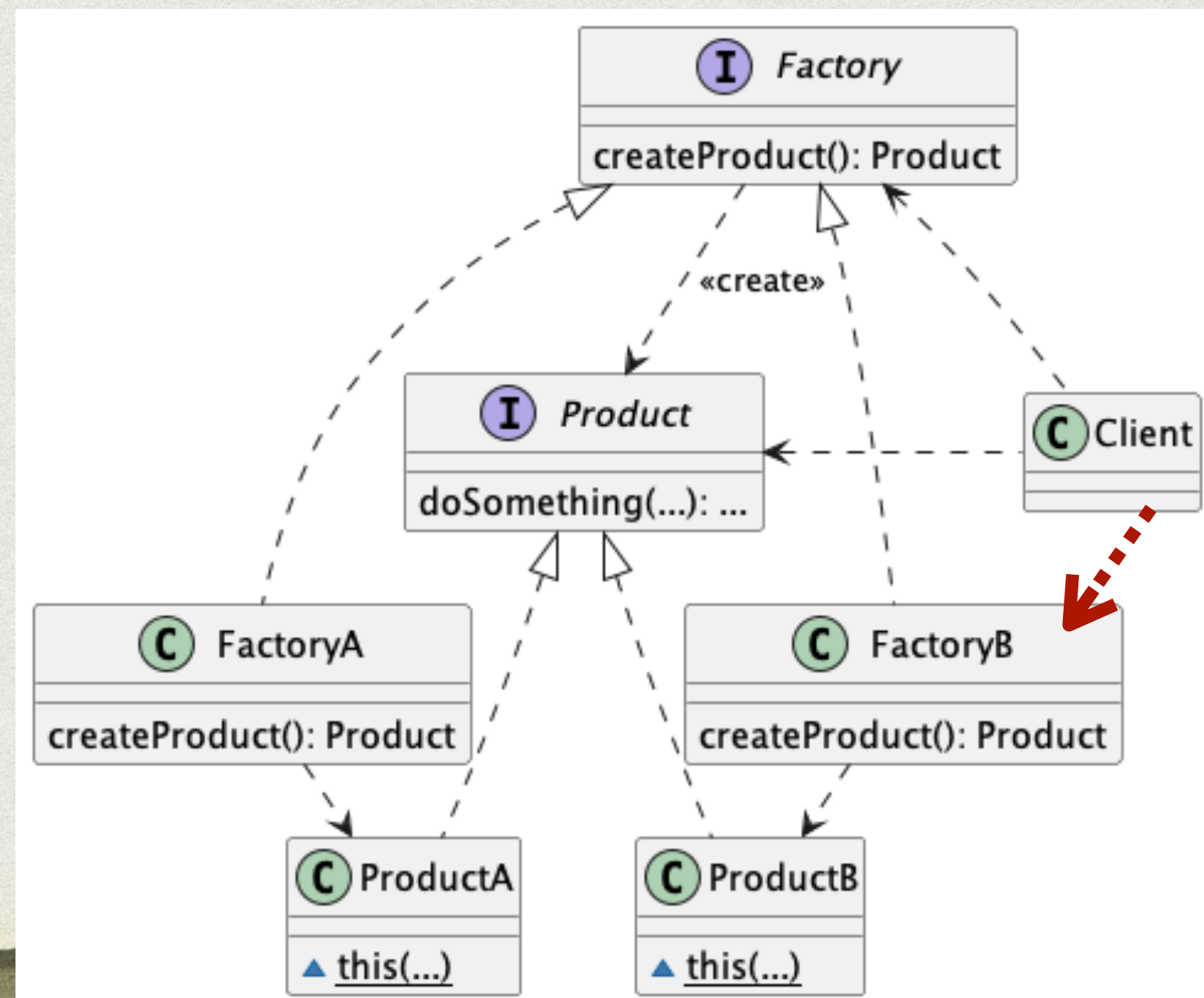
```
public class Deck implements CardSource {
    public Deck() {...}
}
```

couplage client / produit concret

```
// accès direct au constructeur
CardSource source = new Deck();
Card carte = source.draw();
```


FACTORY METHOD

- on a vu des factory methods
- il y a aussi un patron **Factory Method**
interface de création d'objets
+ sous classes raffinant ce qui est créé



```
public interface CardSourceCreator {
    CardSource createCardSource();
}
```

factory spécialisée

interface factory

```
public class Croupier
    implements CardSourceCreator {
    @Override public CardSource createCardSource() {
        return new Deck();
    }
}
```

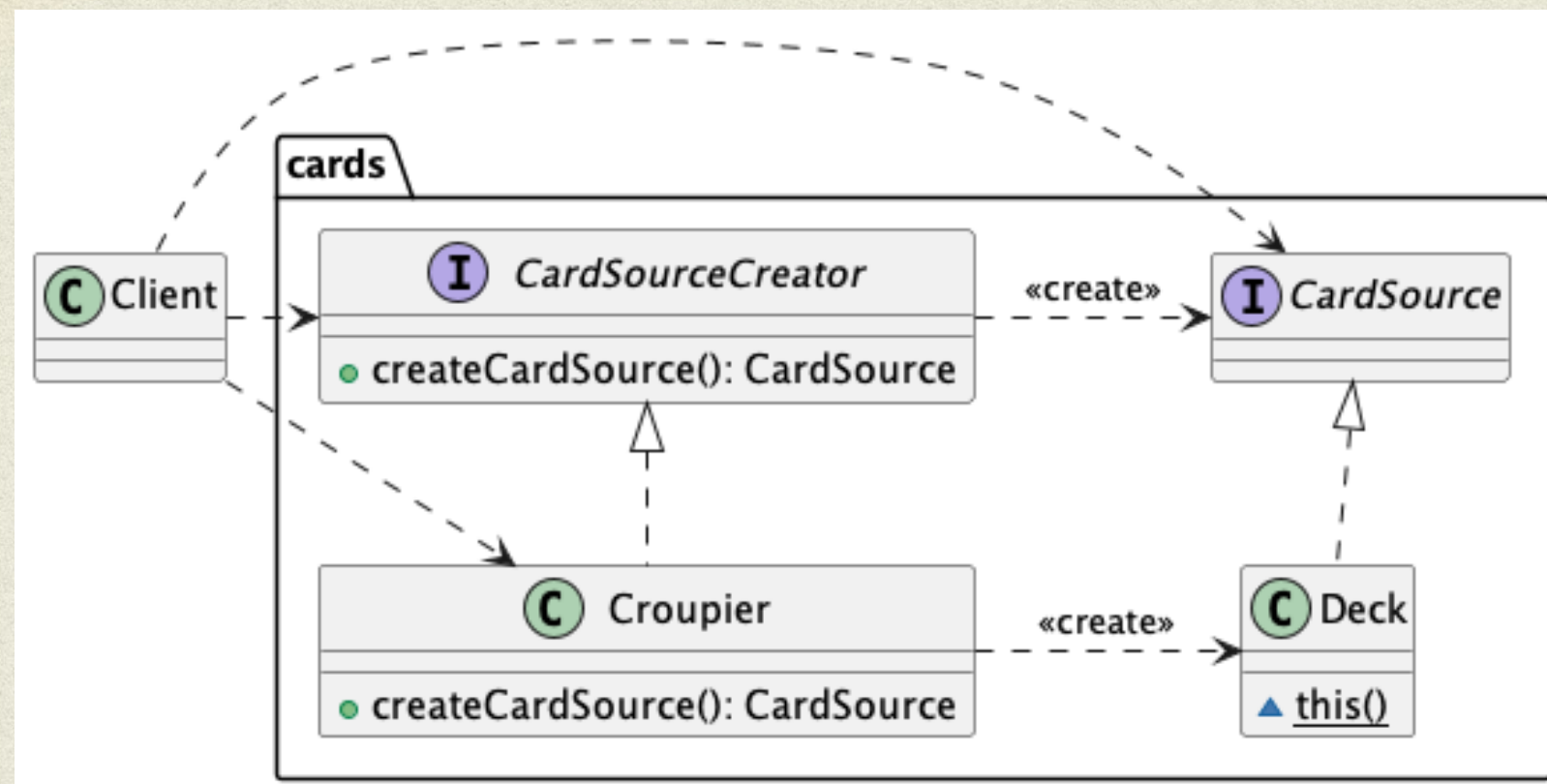
constructeur à accès limité

```
public class Deck implements CardSource {
    Deck() {...}
}
```

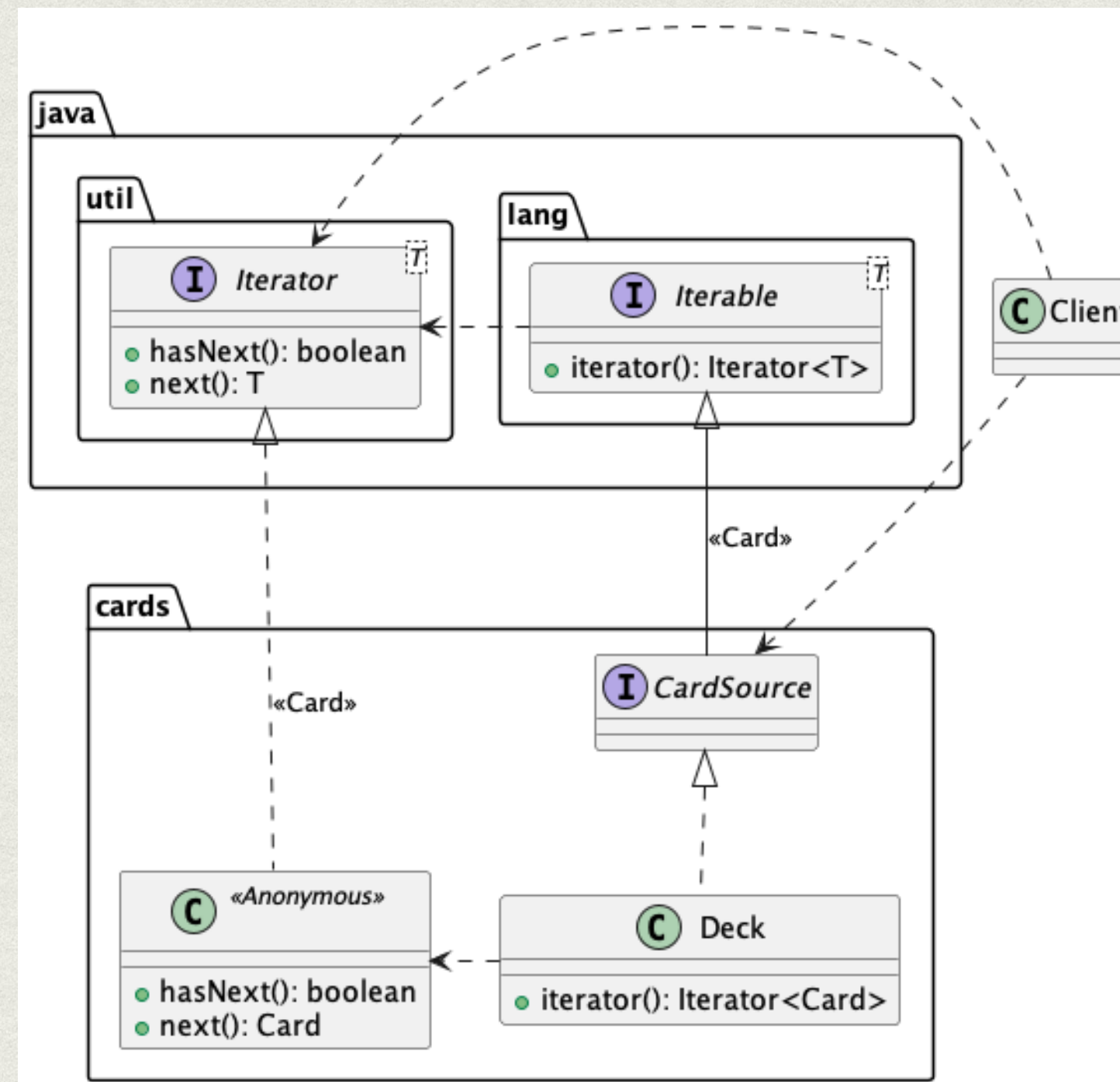
découplage client / produit concret

```
// illustration factory
CardSourceCreator creator = new Croupier();
CardSource source = creator.createCardSource();
Card carte = source.draw();
```

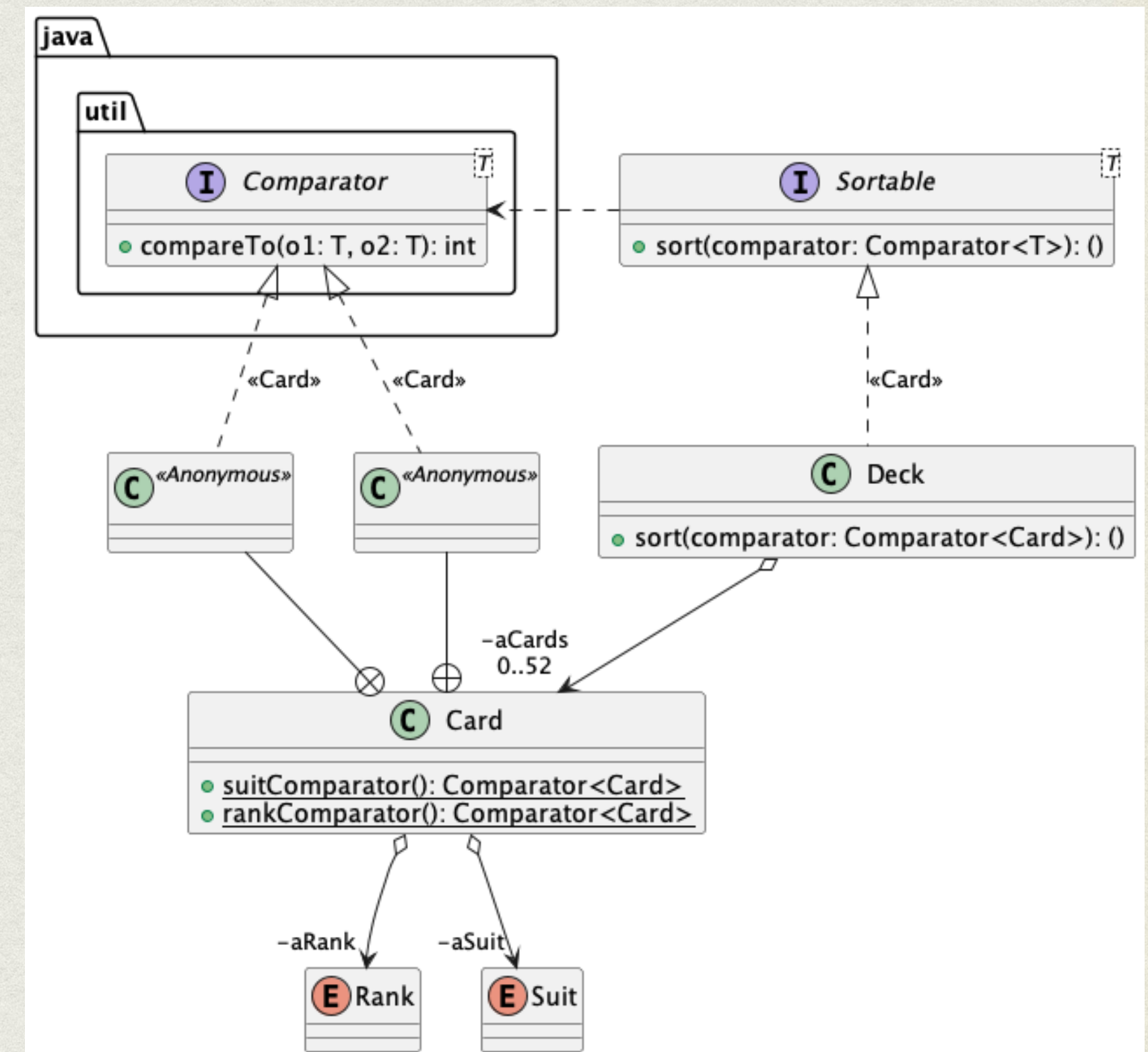

SYNTHÈSE



Factory Method



Iterator



Strategy

SYNTHÈSE

- **découpler** clients / implémentations avec des **interfaces**
appliquer **ISP** (+ sous-typage interfaces)
penser aux interfaces disponibles (*-able)
- patron **Iterator**
exposer des collections en respectant l'encapsulation
- attention au **Switch Statement**⁺
- patron **Template Method**
algorithme avec étapes particulières raffinées, choix statique
- patron **Strategy**
dépendance à une famille d'algorithmes, choix dynamique
- patron **Factory Method**
interface de création d'objets (+ factory methods en général)

